

Day 13 — Generative AI & LLM Fundamentals

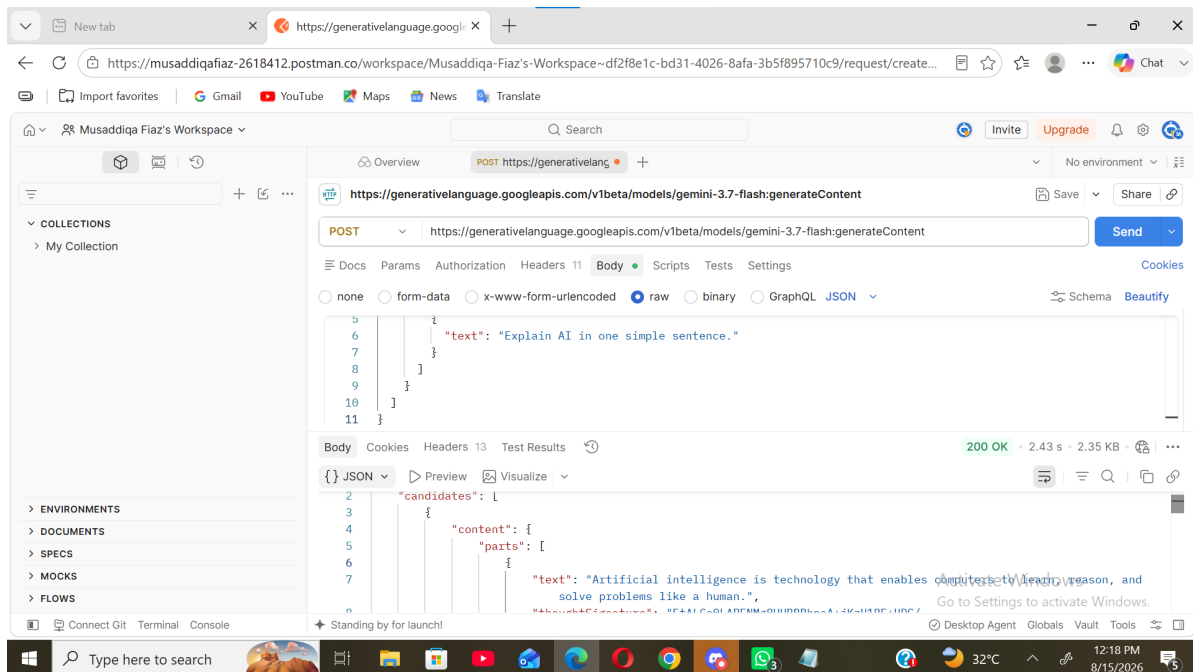
Practical Task Deliverable

Intern Name	Musadiqa Fiaz
University / Institute	Hazara University
Internship Domain	AI Automation
Mode	Online / Remote
Organization	Petalnex Pvt. Ltd.
Supervisor / Team Lead	Sohail Anwar

Proof of Successful API Test Calls

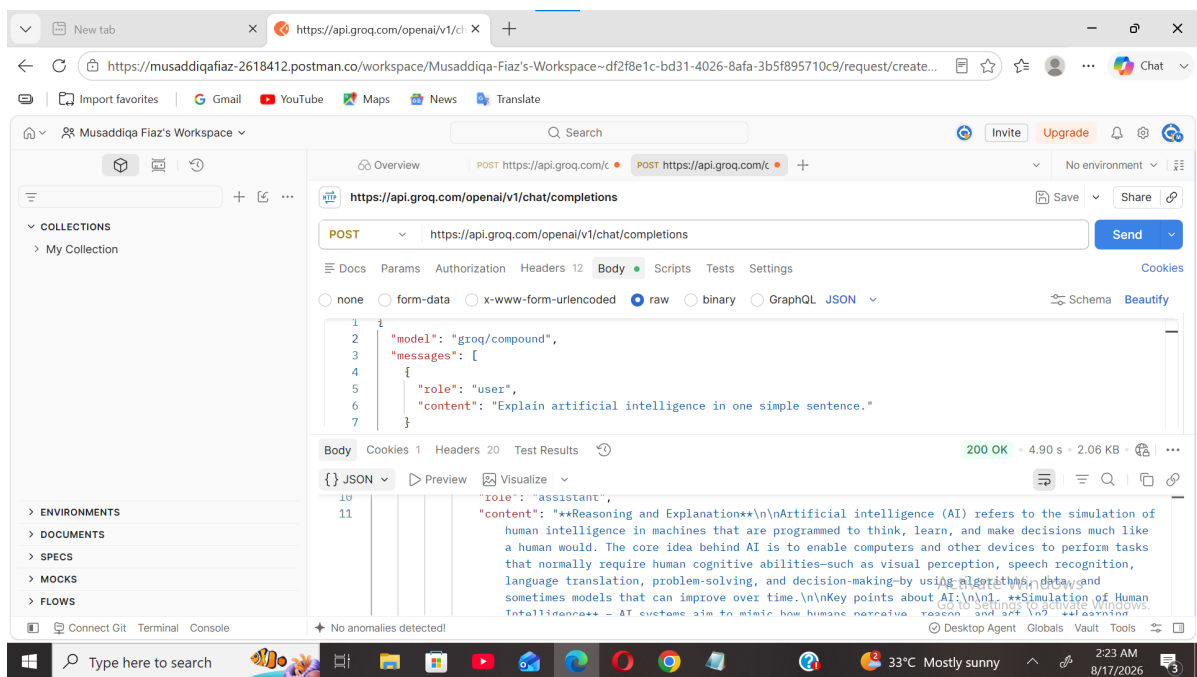
Below are Postman screenshots showing successful test calls to both the Gemini and Groq APIs, each returning a 200 OK response with a generated text reply.

1. Gemini API — Google AI Studio



POST request to the Gemini 2.5 Flash generateContent endpoint returned a 200 OK in 2.43s with a valid candidate response.

2. Groq API — OpenAI-compatible endpoint



POST request to `/openai/v1/chat/completions` using the `groq/compound` model returned a 200 OK in 4.90s with a valid assistant response.

Provider Comparison Note

Classification tasks (intent detection, sentiment tagging, routing short inputs): Groq is the better fit. Groq runs open models (Llama, Mixtral, etc.) on custom LPU hardware, so latency is dramatically lower than typical API calls — often under a second, as reflected in the 2–5 second round trips seen above even including network overhead. Classification prompts are short and don't need deep reasoning, so speed and low cost per call matter more than raw capability. Groq's free tier is generous enough for prototyping this kind of workload at volume, which matters for automation pipelines that fire many small calls.

Long-context work (summarizing large documents, RAG over big knowledge bases, multi-turn analysis): Gemini is the better fit. Google's Gemini models support very large context windows — into the hundreds of thousands of tokens depending on the variant — which lets an entire document or transcript be passed in without chunking. Groq's hosted open models typically have much smaller context windows, so they run into truncation issues on genuinely long documents.

Precision-critical structured output: for tasks where instruction-following and low hallucination matter most — e.g. structured data extraction feeding directly into downstream automation — Claude tends to give the most consistent, schema-bound output, even though it isn't part of the free tiers used in this exercise.

Rule of thumb: cheap + fast + short input → Groq. Big context, needs to "read" a lot → Gemini. Precision-critical structured output → Claude.